

Nexus Financial Gateway API Guide

API INTEGRATION AND AUTHENTICATION GUIDE

Target System: Nexus Financial Gateway Document Version: 1.2 Last Updated: August 18, 2026

1. Authentication (OAuth 2.0) All requests to the Nexus Financial Gateway API must be authenticated using the OAuth 2.0 protocol. Clients must first obtain a bearer token from the `/v1/auth/token` endpoint using the Client Credentials grant type.

To acquire a token, send a POST request with your `client_id` and `client_secret` encoded via Basic Auth in the header. The token response will include an `expires_in` value (typically 3600 seconds). You must attach this token as a Bearer token in the `Authorization` header of all subsequent API requests.

2. Rate Limiting To ensure system stability, the API enforces strict rate limiting. Standard API keys are restricted to 100 requests per minute per endpoint. If you exceed this threshold, the API will return an HTTP 429 Too Many Requests status code. The response header will include an `X-RateLimit-Reset` value indicating the UNIX timestamp when your quota will be replenished.

3. Webhook Subscriptions We highly recommend utilizing our asynchronous Webhook system for transaction status updates rather than polling the `/v1/transactions/{id}` endpoint. To subscribe, submit a POST request to `/v1/webhooks` with a valid HTTPS callback URL and the event types you wish to monitor (e.g., `transaction.cleared`, `transaction.failed`). For security, all webhook payloads sent to your callback URL will include an `X-Signature` header. This is an HMAC-SHA256 hash generated using your secret Webhook Key and the raw request body. You must verify this signature upon receipt to prevent replay attacks.

4. Error Handling The API uses standard HTTP status codes. A 400 Bad Request indicates a malformed payload or missing required fields. A 401 Unauthorized indicates an expired or invalid bearer token. All error responses include a JSON body containing a specific `error_code` and a human-readable `message` to assist with debugging.